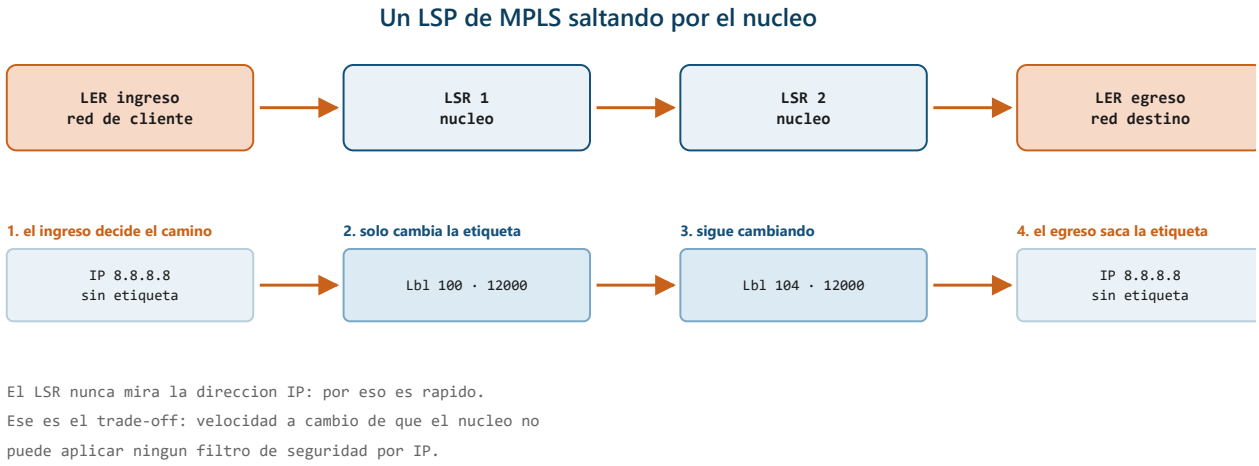
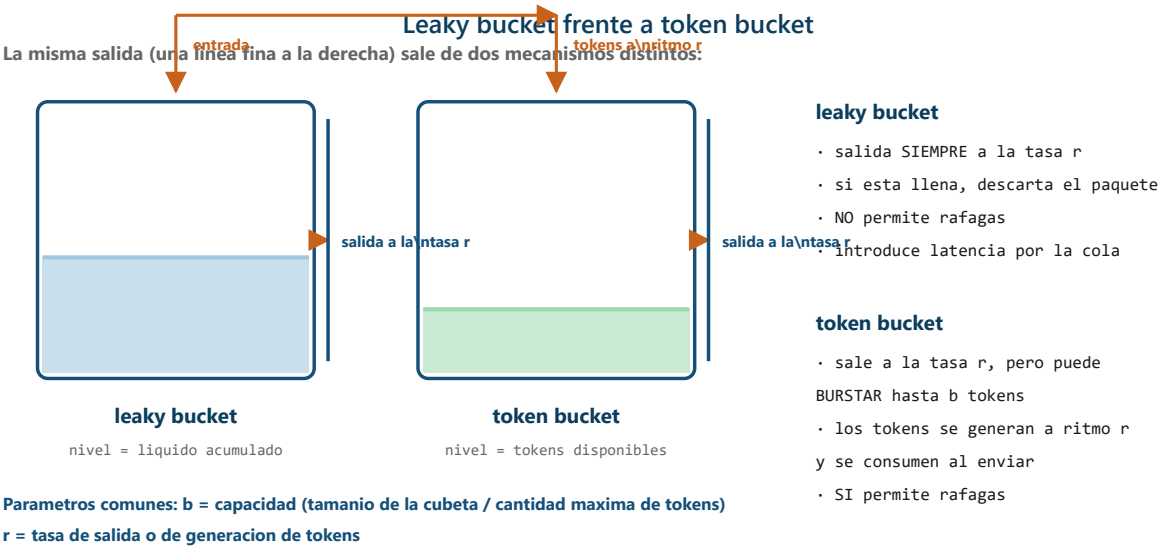
Clase 45 — QoS: shaping, policing, DiffServ y MPLS



Bloque: F (Servicios, multimedia y seguridad) · **Semana:** 15 · **Duración estimada:** 105 min **Correlativa:** Clase 44 (Multimedia: streaming, RTP y VoIP) · **Sigue con:** Clase 46 (Gestión de redes: SNMP, NETCONF y YANG) **Material de referencia:** Tanenbaum cap. 5 §5.6 (control de congestión, control de calidad de servicio) · RFC 2211/2212/2213, RFC 2474, RFC 2475, RFC 3246, RFC 3031, RFC 3032

🎯 Al terminar esta clase vas a poder

1. Explicar por qué Internet es **best effort** y por qué eso rompe las aplicaciones multimedia.
2. Explicar el algoritmo de **leaky bucket** y el de **token bucket**, con sus dos parámetros, y decir **cuál descarta y cuál espera**.
3. Distinguir **shaping** de **policing**, y justificar la arquitectura **ingress policing + egress shaping**.
4. Comparar **IntServ** y **DiffServ**, y explicar **por qué DiffServ ganó**.
5. Dibujar el **byte TOS/DS de IPv4** con los 6 bits de DSCP y los 2 de ECN, y nombrar los **PHB** que hay que saber.
6. Explicar **MPLS**: qué son la etiqueta y el LSP, y **qué hace cada uno de los tres roles de router**.
7. **Seguir el flujo de un paquete** por un LSP, cambiando la etiqueta en cada router.



> **Prerequisito del examen:** `2007-12 P1` (10 pts, leaky bucket vs. token bucket), `2009-03 P4` (10 pts, DiffServ), `2010-07 P4` (8 pts, DiffServ), `2007-12 P4` (10 pts, fragmentación transparente vs. no transparente), `2007-12 P5` y `2006-08 P5` (5 pts cada una, MPLS).

PARTE 1 — TEORÍA

1.1 El problema: Internet no garantiza nada

Todo lo que estudiaste hasta ahora funciona sobre una misma premisa:

```
INTERNET ES "BEST EFFORT" (mejor esfuerzo)
──────────────────────────────────────────

El router reenvía lo que le llega. Si el buffer de salida está lleno,
DESCARTA. Si no, reenvía. No hay nadie que pregunte "¿esto es
importante?".

Consecuencia:
una videollamada de 2 Mbps y una descarga de 2 Mbps
viajan EXACTAMENTE IGUAL.
La descarga gana (es un flujo TCP que insiste), y la videollamada
se rompe.

Y el problema se agrava porque el tráfico no se reparte solo:
el tráfico best effort de una fuente que no implements control
de congestión PUEDE sobre el resto.
```

El ejemplo que hay que tener en la cabeza: en una red donde hay un enlace de acceso de 10 Mbps, tenés una descarga de 10 Mbps y una videollamada de 1 Mbps. El **ancho de banda alcanza** (11 > 10 es falso, pero imaginá 20 Mbps de enlace y 10+1). Sin embargo la videollamada se entrecorta, porque:

```
El BUFFER del router, no el ancho de banda, es el problema.
──────────────────────────────────────────────────────────

El paquete de voz llega, encuentra su cola de 100 paquetes llena
(because la descarga llenó el buffer), y se descarta.

PÉRDIDA de 1 paquete de voz = 20 ms de la palabra del otro.

Y el agregado es peor: la descarga ve la pérdida, baja cwnd, y la
videollamada ve la pérdida y no puede hacer nada (no retransmite).
```

Y hay un segundo problema, más sutil: la **cola grande** en los routers es en apariencia una mejora, y en realidad es una trampa. Se llama **bufferbloat**:

```
Buffer PEQUEÑO (1 paquete de cola)
→ se pierde rápido, el router no se atrasa mucho
→ throughput bajo, LATENCIA BAJA

Buffer GRANDE (1000 paquetes de cola)
→ se pierde tarde, el router "aguanta" todo
→ throughput alto, LATENCIA ENORME (el paquete espera 2 s)

El búfer grande convierte una pérdida de 0,1 % en 2 segundos de
latencia. Para el tráfico interactivo, el bufferbloat es PEOR que
la pérdida.

⇒ La calidad de servicio NO es un lujo: es la única herramienta que
tenemos contra el bufferbloat.
```

1.2 Los dos algoritmos de control de caudal: leaky bucket y token bucket

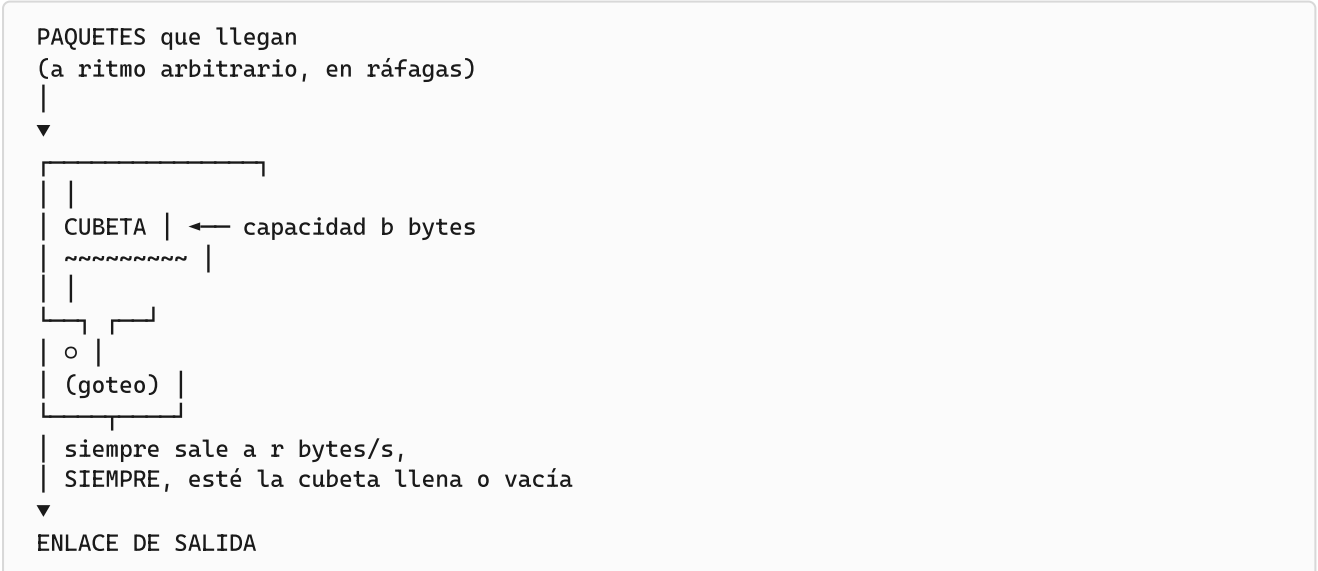
Son los dos mecanismos básicos de **conformado de caudal** (*rate shaping*), y son de capa 2, de capa 3 o de un router. Los dos se usan para decir: "de este enlace no puede salir más de `r` bytes por segundo".

(a) Leaky bucket — cubeta con goteo

El nombre viene de la imagen, y es buena imagen: una cubeta con un agujero delgadito en el fondo. El agua entra a ritmo arbitrario y sale **siempre al mismo ritmo**, por el agujero.

```
Los 3 parámetros (y hay que memorizarlos con estos nombres):
─────────────────────────────────────────────────────────────
b = capacidad de la cubeta, en BYTES (el tamaño del buffer)
r = tasa de salida, en BYTES POR SEGUNDO (el agujero)
f = tiempo de vaciado = b / r (cuánto tarda la cubeta en vaciarse)

El tercer parámetro es el que se pregunta menos y el que explica
todo: f es la "ventana de compatibilidad" con el jitter.
```



El algoritmo, literal:

```
CUBETA VACÍA
LLEGA un paquete de tamaño L:

SI (contenido + L) ≤ b : contenido ← contenido + L (aceptado)
SINO: DESCARTAR el paquete (o marcarlo)

PERIÓDICAMENTE (cada 1/r segundos):
contenido ← contenido − r (o contenido − L del primero)
contenido nunca baja de 0 (si baja, la cubeta está vacía y
no sale nada)
```

El detalle que hay que entender bien: aunque la cubeta esté vacía, **el goteo no se detiene**: sigue saliendo un paquete cada `1/r` segundos. Eso es lo que produce el **flujo constante** (salida a ritmo constante) aunque la entrada sea a ráfagas. El precio es que, si la cubeta se vacía, se **desperdicia capacidad**: el enlace queda libre aunque haya tráfico esperando.

Con números (esto es lo que hay que hacer en el examen):

```
b = 100 bytes, r = 10 bytes/s, paquetes de 20 bytes.

Cuántos paquetes de 20 bytes entran en 1 segundo?

── ¿Cuántos ENTRAN? (durante 1 s la cubeta recibe lo que le llega)
Esto depende de la tasa de ENTRADA, que el enunciado tiene que dar.
Si la entrada es de, digamos, 30 paquetes/s:
contenido inicial = 0
t = 0,00 : llega P1 (20) → contenido = 20 ≤ 100 ✓
t = 0,03 : llega P2 (20) → contenido = 40 ✓
t = 0,07 : llega P3 (20) → contenido = 60 ✓
t = 0,10 : llega P4 (20) → contenido = 80 ✓
t = 0,13 : llega P5 (20) → contenido = 100 ✓ (llena)
t = 0,17 : llega P6 (20) → 120 > 100 ✗ DESCARTA
(y todos los que siguen hasta que se vacíe)

En 1 s, de una entrada de 30 paquetes/s (30 paquetes), entran
5 y se descartan 25.

── ¿Cuántos SALEN? (esto sí lo determina el algoritmo)
El goteo es de 10 bytes/s y cada paquete pesa 20 bytes
⇒ sale 1 paquete cada 20/10 = 2 segundos.

En 1 s salen 0,5 paquetes. Ojo: en 1 segundo puede no salir NINGUNO.
Con f = b/r = 100/10 = 10 s, la cubeta tarda 10 s en vaciarse.

RESUMEN DEL EJEMPLO:
el leaky bucket convierte UNA ENTRADA A RÁFAGAS en una SALIDA
A RITMO CONSTANTE de 10 bytes/s, y ELIMINA todo lo que exceda.
```

(b) Token bucket — cubeta con tokens

La diferencia es **una sola palabra**: en vez de agua, hay **tokens** que se producen a ritmo constante y se **consumen** al mandar paquetes.

```
PAQUETES que llegan
(a ritmo arbitrario)
|
▼
┌─────────────────┐
| o o o o o | ←── los TOKENS se generan a r tokens/s
| o o o o o | (hasta un máximo de b tokens)
| o o o o o |
└─────────────────┘
|
para enviar un paquete de L bytes:
NECESITO L tokens. Si no hay L, el paquete ESPERA.
|
▼
ENLACE DE SALIDA
```

El algoritmo, literal:

```
GENERADOR (independiente de los paquetes, corre siempre):
cada 1/r segundos: tokens ← min(tokens + 1, b)
(tokens nunca supera b: la cubeta de tokens tiene capacidad b)

LLEGA un paquete de tamaño L:

SI tokens ≥ L : tokens ← tokens − L ; ENVIAR el paquete
SINO : ESPERAR hasta que haya L tokens, y recién
ahí enviarlo.
```

El contraste con el leaky bucket, que es lo que más se pregunta:

	LEAKY BUCKET	TOKEN BUCKET
Qué produce	tokens	tokens
Qué se genera	NADA	r tokens/s
Qué pasa si no hay "crédito"	se DESCARTA el paquete (o se marca)	el paquete ESPERA (shaping)
Ritmo de SALIDA	SIEMPRE constante	puede tener RÁFAGAS
Ráfagas	IMPOSIBLES	POSIBLES (hasta b)
Utiliza toda la capacidad si sobra	NO (se desperdicia)	SÍ
¿Retrasa?	Sí, inherentemente (por el buffer)	Sí, si el credit no alcanza
Uso típico	control de caudal "duro"	control de caudal "blando"

La frase que hay que saber decir: *"el leaky bucket descarta, el token bucket espera; el leaky bucket produce flujo constante, el token bucket permite ráfagas de hasta* `b`*."*

Y el caso límite que hay que entender: si el token bucket tiene `b` tokens acumulados y entran `b` paquetes de golpe, **salen los `b` juntos**: eso es una ráfaga. Con el leaky bucket eso es imposible, por construcción.

```
TOKEN BUCKET, r = 10 tokens/s, b = 50 tokens, paquetes de 10 bytes:

t=0 : tokens = 50 (lleno) llegan 5 paquetes de 10 bytes
P1: tokens 50≥10 → tokens=40, sale
P2: tokens 40≥10 → tokens=30, sale ← los 5 salen
P3: tokens 30≥10 → tokens=20, sale ← SEGUIDOS.
P4: tokens 20≥10 → tokens=10, sale Eso es una
P5: tokens 10≥10 → tokens= 0, sale ← RÁFAGA de 50 bytes
t=0.1 : tokens = 0 + 1 = 1
t=0.2 : tokens = 2 ... y así, se va reponiendo a 10 tokens/s

Con leaky bucket (b=50, r=10, paquetes de 10):
t=0 : entra P1 (10 ≤ 50) cubeta=10 ; P2 cubeta=20 ; P3=30 ; P4=40 ; P5=50
sale 1 paquete cada 10/10 = 1 s. Los 5 salen a t=1,2,3,4,5.
NUNCA se pueden salir juntos. ← misma cubeta, resultado opuesto.
```

1.3 Shaping vs. policing: los dosWhere se usan

```
SHAPING (conformado) POLICING (vigilancia)
──────────────────── ──────────────────────

ENCOLA y RETRASA DESCARTA o REMARCA lo que excede
←── buffer + scheduler ──→ ←── sin buffer, sin retardo ──→
Mantiene la tasa, deforma el timing Mantiene la tasa exacta,
(introduce LATENCIA) no deforma nada
El paquete SIEMPRE pasa (tarde) El paquete NO pasa
Implementado con: token bucket + cola Implementado con: token bucket
solo de conteo
```

¿Por qué los dos? Porque cada problema tiene su lugar natural:

```
ENLACE (o router) por qué ese
──────────────────────────────────── ──────────────────────────

ENTRADA (ingress) POLICING
Un usuario no autorizado podría No querés RETRASAR un paquete
mandar 100 Mbps yaybe le ocupes que venía de una app legítima
todo el buffer del router. (¡es una discriminación!).
→ Tenés que tira algo. Además, meterlo en cola te
→ Y sin latencia: no tenés derecho. agrega latencia a un paquete
que ya llegó tarde.

SALIDA (egress) SHAPING
Acá SÍ sos vos el dueño del enlace. Querés aprovechar al máximo el
Los paquetes son tuyos y de tus enlace y absorber las ráfagas
usuarios: podés priorizar, (p. ej. hacia un enlace de
delaying y descartar. bajada más lento que el de
→ Podés jugar con la latencia. subida).
```

De ahí la frase estándar de la industria:

> **La arquitectura clásica es "policing en la entrada, shaping en la salida".**

1.4 IntServ vs. DiffServ

IntServ — el modelo que prometía garantías y no escaló

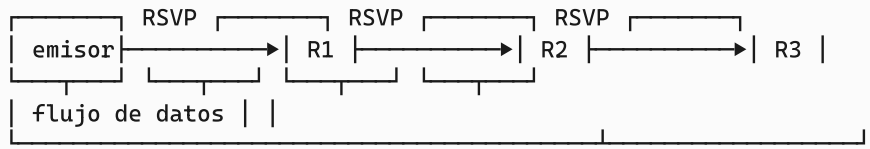
Integrated Services (`RFC 2211`, `RFC 2212`, `RFC 2213`, 1997). Es el modelo "de lujo": **reserva recursos por flujo**.

```
REQUISITOS:
1. El emisor (o un "controlador de admisión") dice QUÉ calidad quiere
para un flujo: "necesito 2 Mbps, 20 ms de retardo, jitter < 5 ms".

2. Se señaliza con RSVP (Resource ReSerVation Protocol) UNA RUTA COMPLETA:
RSVP Hello → RSVP Request → RSVP Confirm (de abajo hacia arriba)
y el router decide: "¿tengo ancho de banda y buffer para esto?"

3. Si los routers de la ruta ACEPTAN, se RESERVA: ancho de banda en
cada enlace, buffers con tamaño fijo, y se instalan clasificadores
y marcadores de tiempo en cada router.

4. Ahora el flujo tiene GARANTÍA: bw mínimo, retardo máximo, jitter
máximo, tasa de pérdida acotada. Es un CIRCUITO VIRTUAL con garantías.
```



(con la reserva ya instalada)

El problema fatal, y es lo que hay que decir en el examen:

```
Cada router debe GUARDAR ESTADO POR FLUJO:
- la reserva de ancho de banda de cada flujo en cada enlace
- un clasificador que mire CADA paquete para decidir a qué flujo
pertenece (5-tupla) y aplicarle la QoS correcta
- un buffer por clase

Coste: memoria y CPU CRECEN con el número de flujos.

Cantidad de flujos simultáneos en Internet (2026):
YouTube: ~millones
videollamadas: ~cientos de miles simultáneas
Spotify: ~cientos de millones de usuarios

Un router de backbone no puede guardar estado de millones de flujos.
⇒ IntServ NO ESCALA. Punto.

Además tiene otros problemas:
- el problema del "control distribuido": si un router del path se
cae, hay que renegociar toda la reserva
- el emisor tiene que conocer la topología
-.movie el "diamante": si hay dos caminos posibles, hay que reservar
en AMBOS (o quedarse sin garantías)
```

> **Actualización a 2026:** IntServ ≈ **no se despliega a escala**. La única supervivencia real es **RSVP-TE** (`RFC 3209`) para *traffic engineering* de MPLS, donde el número de flujos de tráfico reservado es **ordenes de magnitud menor** que en Internet. Decilo así: *"IntServ es el modelo que no escaló, y DiffServ el que ganó"*.

DiffServ — el modelo que escaló

Differentiated Services (`RFC 2474` el campo DS, `RFC 2475` la arquitectura). La idea es **voltear el modelo**: en vez degarantía por flujo, se garantiza por **CLASE**.

```
LA INVERSIÓN FUNDAMENTAL:
─────────────────────────
IntServ: señalización explícita + estado POR FLUJO + garantías
DiffServ: NINGUNA señalización + estado POR CLASE + clases de servicio

"En vez de reservar para cada llamada, reservo para 'vocoder audio'."
```

El modelo tiene tres pasos, y el **nombre de cada uno hay que memorizarlo**:

```
1. CLASSIFICATION (clasificación) en el EDGE (borde)
El router del borde CLASSIFICA el flujo. Usa una tabla
(puerto, protocolo, IP, DSCP) → DSCP. Ejemplo:
(RTP, puerto 16384-32767) → DSCP EF (voz)
(HTTP, puerto 80) → DSCP AF11 (web)
(todo lo demás) → DSCP 0 (best effort)

2. MARKING (marcado)
El borde ESCRIBE el DSCP en el byte TOS del paquete IP.
El DS field viaja por Internet.

3. PHB (Per Hop Behaviour) en el CORE (núcleo)
Cada router del core aplica la política correspondiente al DSCP
que ve. NO necesita señalización. NO necesita saber qué flujo es.
Sólo necesita leer 1 byte.

RESUMEN EN UNA FRASE (la que hay que escribir):
─────────────────────────────────────────────────
"El EDGE clasifica y marca; el CORE encamina aplicando el PHB
definido para cada marca, usando el campo TOS de IPv4."
```

La diferencia conceptual que hay que entender (es lo que se pregunta):

```
IntServ DiffServ
──────── ─────────
Garantía POR FLUJO Garantía POR CLASE
(este flujo, estos parámetros) (todos los de voz se tratan igual)
identidad del flujo = 5-tupla identidad = 6 bits de DSCP
señalización: RSVP señalización: NINGUNA
estado: en cada router estado: en cada router, pero SOLO
por clase (unas pocas, 12-ish)
escala: NO escala: SÍ
```

La analogía que ayuda: DiffServ es como un **sala de espera con dos categorías** ("los de la fila A" y "los de la fila B") en lugar de un ticket individual con turno exacto para cada persona. Es más simple, más justo en promedio, y escala.

⚠️ Actualización a 2026

Sigue igual, de memoria: - **DSCP / PHB**: **EF** (Expedited Forwarding, `RFC 3246`, para voz), **AF11..AF43** (`RFC 2597`, cuatro clases × tres drop precedences), **Default/BE**, y **CS** (Class Selector, heritage de los valores de TOS anteriores a DSCP). - **Marking en el borde + PHB en cada hop.** - **Policing vs. shaping**: policing con token bucket (descarta o remarkea lo que excede, sin buffer); shaping con cola + token bucket (retrasa y suaviza, introduce latencia). **Ingress policing + egress shaping. Esto no cambió.** - `RFC 8622` (2019) actualiza al PHB **LE** (*Lower Effort*, *Lower Expected effort*) y **obsolesce** `RFC 3662`, que era el PDB de DiffServ. Si el enunciado cita `RFC 3662`, está desactualizado.

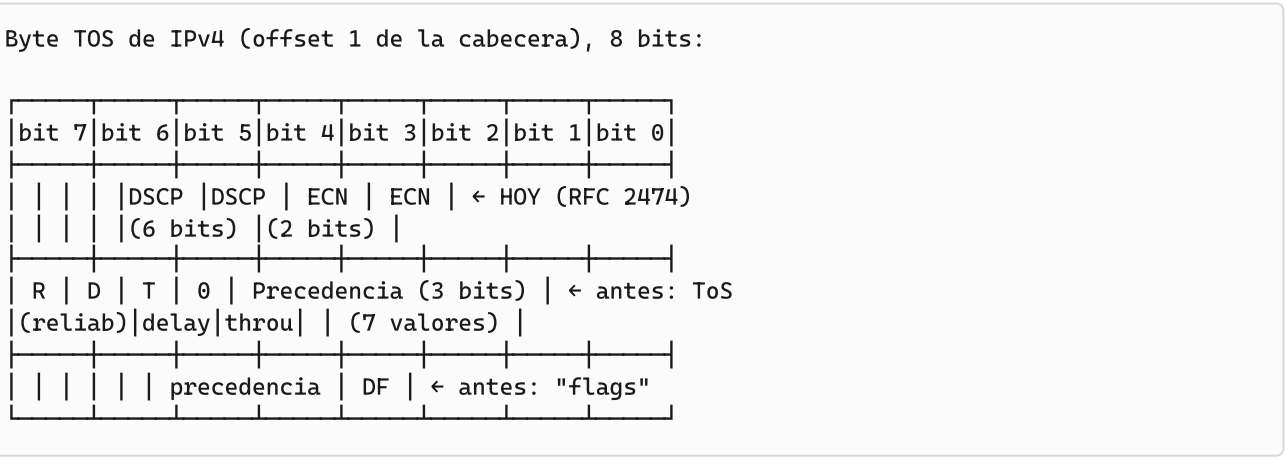
Cambió: - **DiffServ ganó hace más de 20 años.** No hay nada que actualizar: es el modelo desplegado. - Lo que **sí** es nuevo es el **equilibrio (ECMP)**: con unequal-cost load balancing, el PHB puede dejar de funcionar bien, porque el camino cambia a cada salto y los buffers no están donde el modelo los pone. En la práctica hoy se combina ECN + AQMs en vez de DiffServ puro.

1.5 El campo de marcado: el byte TOS / DS de IPv4

Este es **el detalle fino que más se pregunta** (`2009-03 P4` y `2010-07 P4`).

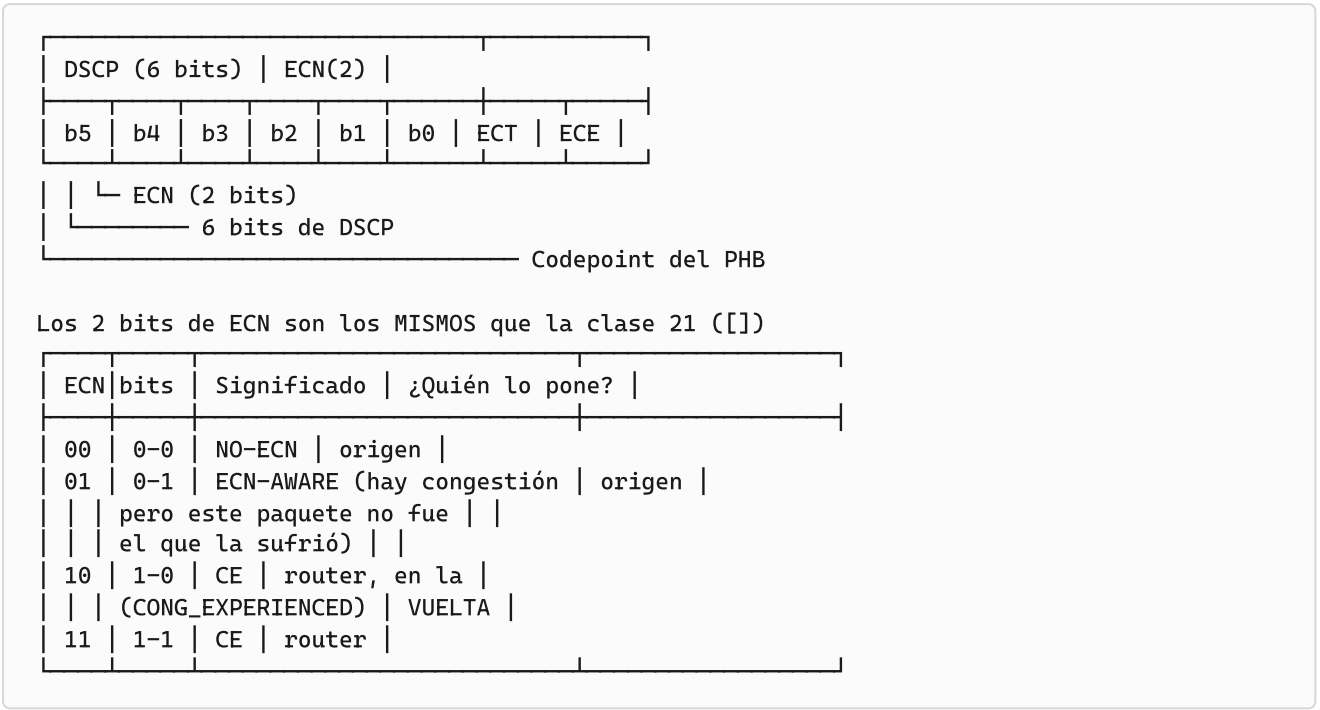
La historia del byte, que es lo que hay que saber

El byte de tipo de servicio de IPv4 tiene 8 bits y **se usó para tres cosas distintas** a lo largo del tiempo. Por eso los nombres son confusos:



> **En el examen, cuando el enunciado dice "el campo TOS de IPv4", la respuesta moderna es: el byte DS (*Differentiated Services*), con 6 bits de DSCP y 2 bits de ECN.** No la precedencia de 3 bits ni los bits R/D/T.

El layout del byte DS (el que hay que dibujar)



Los 2 bits de ECN son los MISMOS que la clase 21 ([])

ECN	bits	Significado	¿Quién lo pone?
00	0-0	NO-ECN	origen
01	0-1	ECN-AWARE (hay congestión pero este paquete no fue el que la sufrió)	origen
10	1-0	CE (CONG_EXPERIENCED)	router, en la VUELTA
11	1-1	CE	router

Regla de oro para el examen: el bit de ECN va después del DSCP, y no al principio. El error típico es ponerlos al revés porque el layout "histórico" tenía la precedencia primero.

Los PHB que hay que saber

```
PHB DSCP Para qué
──────────────────────────── ──────── ──────────────────────────────────────
Default / Best Effort 000000 Sin ningún tratamiento especial. Es el
caso por defecto.
──────────────────────────────────────────────────────────────────────────────
EF (Expedited Forwarding) 101110 Voz. RFC 3246. Es un "tubería": muy
(46 binario = 0b101110) baja probabilidad de descarte, banda
DSCP = 46 (11 de 6) mínima garantizada, y **NO se permite
encolarlo detrás de nada**. Se usa
con policing/shaping estricto: si hay
más voz de la que cabe, la bajan.
──────────────────────────────────────────────────────────────────────────────
AFxy (Assured Forwarding) AF11 4 clases (1..4) × 3 drop precedences
AF11 .. AF43 AF12 (1 = menor, 3 = mayor). AF1 = tráfico
DSCP: AF11=001010 (10) AF13 interactivo (voz, juegos), AF2 = FTP,
AF12=001100 (12) AF21 AF3 = web, AF4 = bulk (correo, backup).
AF13=001110 (14) AF22 La diferencia entre clases es CUÁNTO
AF21=010010 (18) AF23 ancho de banda se GARANTIZA. La
AF22=010100 (20) AF31 diferencia entre los "y" de una misma
AF23=010110 (22) AF32 clase es la PRIORIDAD de descarte:
AF31=011010 (26) AF33 si hay congestión, se tira primero
AF32=011100 (28) AF41 un AFx3, después un AFx2, después un
AF33=011110 (30) AF42 AFx1, y los AFy de la clase más baja.
AF41=100010 (34) AF43
AF42=100100 (36) AF43=100110 (38)
──────────────────────────────────────────────────────────────────────────────
CS (Class Selector) 1xxxxxx Heritage: los 8 valores que existían
en el byte TOS viejo. CS1..CS7. Hoy
están "reservados" y no se definen
como PHB por RFC. CS0 = 000000 =
Default.
```

Los PHB oficiales con RFC (de `ACTUALIZACIONES.md` §12):

- `RFC 2474` — el campo **DS**
- `RFC 2475` — la **arquitectura** de DiffServ
- `RFC 3246` — **EF** (Expedited Forwarding)
- `RFC 2597` — **AF** (Assured Forwarding)
- `RFC 4594` — **guidelines** de clases de servicio (cómo configurar el conjunto)
- `RFC 8622` — **LE PHB** (obsolesce `RFC 3662`)

1.6 MPLS: etiqueta, LSP y los tres roles

MPLS (Multiprotocol Label Switching) resuelve un problema muy concreto: en IP puro, cada router tiene que **mirar la dirección IP de destino** de cada paquete y hacer una **búsqueda en la tabla** (longest prefix match). Eso es caro en hardware de alta velocidad, y obliga a que el equipo tenga una TCAM enorme.

El label (etiqueta) — 4 bytes

```
┌──────────────────────────────┬──────────────┬─────────┬──────┐
│ Label (20 bits) | TC (3) | S | TTL |
├──────────────────────────────┼──────────────┼─────────┼──────┤
| índice de la FEC a la que | Traffic | Stack | |
| pertenece este paquete | Class | depth | |
└──────────────────────────────┴──────────────┴─────────┴──────┘
▲ ▲ ▲ ▲
| | | └─ se usa IGUAL que
| | | en IP: para el
| | | loop detection y
| | | traceroute
| | └─── S=1 significa que
| | hay otra etiqueta debajo
| └─── TC: 3 bits para QoS/prioridad
└─── "índice en la tabla": dice "este paquete es de la
clase de la FEC X; aplicá el tratamiento de la entrada X"
```

Lo esencial: la etiqueta **no es una dirección**. No contiene el destino, no se puede "enrutar" por ella en el sentido de IP. Es **un índice en la tabla del router**. Y por eso es **una entrada de tabla, no una búsqueda**.

Y el campo TC (Traffic Class) es donde vive el QoS de MPLS: los 3 bits del TC se usan para marcar la prioridad, y hay **diferentes tramas de experimentación** (*EXP`, TC`, Precedence`) según el RFC. Es el equivalente MPLS del DSCP.

La FEC (Forwarding Equivalence Class)

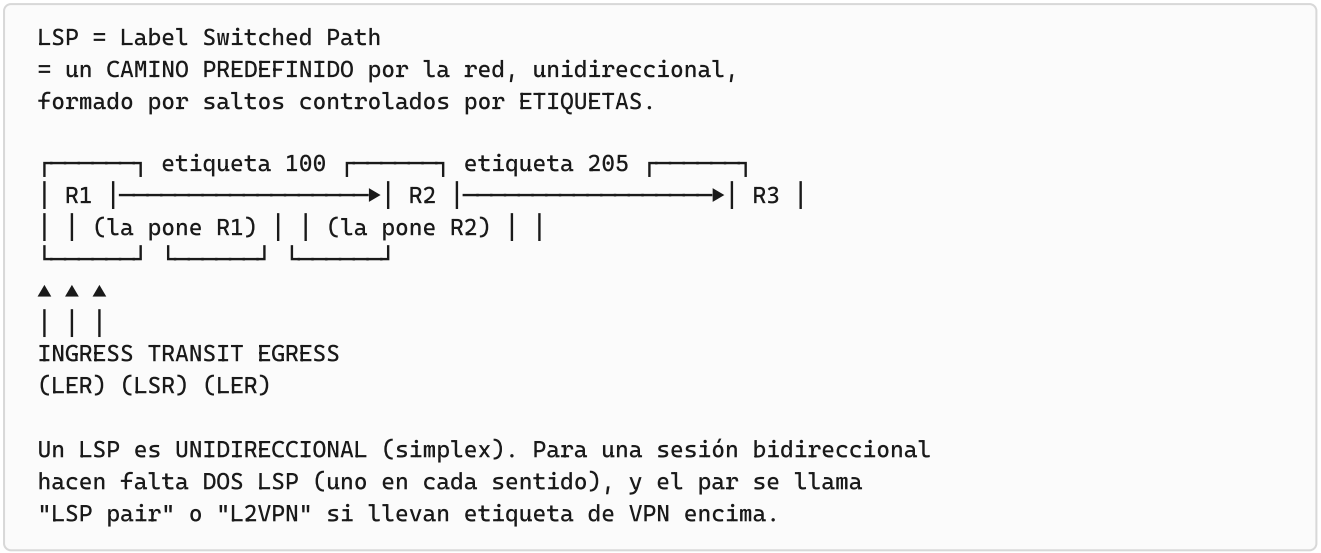
Es el concepto que hay que definir sí o sí:

```
FEC = el grupo de paquetes que receives TRATAMIENTO IDÉNTICO.

En la práctica: "los paquetes que van del prefijo 10.1.1.0/24 al
prefijo 10.2.2.0/24" son una FEC. "Los paquetes con destino 172.16/12
que entran por la interfaz 3" son otra FEC.

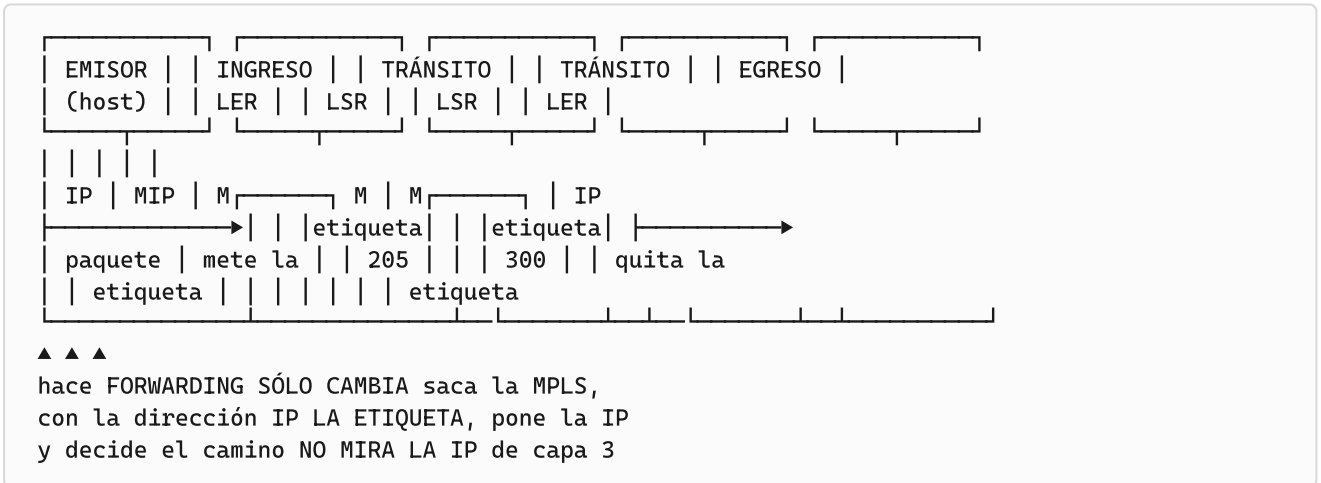
La idea: no me interesa el destino EXACTO de cada paquete. Me
interesa qué grupo al que pertenece. Eso me permite reenviar mirando
20 bits en lugar de 32 (o de una tabla con 1 millón de entradas).
```

El LSP (Label Switched Path)



Y la diferencia con un circuito virtual clásico es la que hay que saber: el LSP de MPLS **no reserva recursos**. Es una "sugerencia" de camino; los recursos los decide el *ingress* router en función de la **tabla de forwarding**. Por eso MPLS **no es un circuito virtual**, y por eso el problema de la clase 25 ("VC reserva recursos") reaparece: MPLS se puede combinar con DiffServ para tener clase de servicio, pero **no** garantiza ancho de banda por flujo.

Los TRES roles de router (esto es lo que preguntan)



Rol	Sinónimos	Qué hace	Qué mira
Ingress (*head-end*)	**LER** (*label edge router*)	**examina** los paquetes IP entrantes, los **clasifica en una FEC**, **decide el camino**, **pone la etiqueta inicial** (hace *push*)	**La dirección IP**
Transit (*transit LSR*)	**LSR**	**reenvía el paquete MPLS cambiando la etiqueta** (*swap*). No consulta la dirección IP.	**Sólo la etiqueta**
Egress (*tail-end*)	**LER**	**quita la etiqueta MPLS** (*pop*) y pone la **cabecera de capa 3** de abajo	La etiqueta

El detalle que hay que subrayar y que es la ganancia de MPLS:

> **El router de tránsito NO mira la dirección IP.** Toma 20 bits de etiqueta, hace una **búsqueda exacta** (o un switch directo por ASIC), y escribe 20 bits nuevos. Eso es lo que hace que MPLS sea **más rápido** que el forwarding por IP: una **búsqueda exacta** en lugar de un **longest prefix match** sobre 32 bits en una TCAM.

Y el otro detalle que se pregunta: en MPLS **se puede hacer forwarding de familias que no son IP**. El nombre completo lo dice: *Multiprotocol* **Label Switching*. Un router de tránsito MPLS** no necesita entender el protocolo que transporta**: reenvía tramas por label y no le importa si es IP, ATM o Ethernet.

Los 4 roles de etiqueta (y acá hay una tabla que hay que saber):

```
Swap = el caso normal: se cambia una etiqueta por otra (tránsito)
Push = se agrega una etiqueta encima (ingreso, o cuando el LSP se
divide por uno de los LSRs)
Pop = se saca la etiqueta de arriba (salida del LSP)
Impose = se saca una etiqueta de una capa inferior y se pone la de la
capa de arriba (penultimate hop popping)
```

⚠️ Actualización a 2026

Sigue igual, de memoria: - **MPLS sigue vivo en el plano de datos de los carriers.** La etiqueta, el LSP, los tres roles, el `TC`, el swap/push/pop: todo igual. - Los RFC: `RFC 3031` (arquitectura), `RFC 3032` (label stack), `RFC 4364` (MPLS VPN).

Cambió — y acá está la noticia fuerte:

- **El plano de control se está moviendo a Segment Routing.** SR (`RFC 8402`, arquitectura) codifica la ruta con **SIDs** (*Segment Identifiers*) en vez de estados por prefijo, y el estado vive **solo en el borde de ingreso**.
- **SR-MPLS** (`RFC 8660`) usa etiquetas MPLS para codificar los SIDs.
- **SRv6** (`RFC 8986`) **elimina el MPLS por completo**: codifica los SIDs directamente en la **cabecera IPv6**, con un encabezado de Routing/IPv6 opcional. Ése es el sucesor real en despliegues nuevos.
- **En datacenter, MPLS fue reemplazado por EVPN-VXLAN** (`RFC 7432` + `RFC 7348`). VXLAN encapsula tramas de Ethernet sobre **UDP/4789**, con lo cual un tenant puede tener su propia tabla MAC y su propio dominio de broadcast. Hoy el overlay de datacenter **no es MPLS**: es EVPN-VXLAN.
- **Por qué MPLS no se muere** (respuesta de examen, es lo que hay que saber): porque MPLS construye **VPNs de capa 3 con aislamiento estricto**: un tenant no ve los prefijos del otro, ni sus direcciones MAC, ni sus rutas. Eso es un requisito de seguridad y de contabilidad que una red de Ethernet compartida **no** da. SRv6 quiere reproducir eso, y lo está haciendo.

1.7 Fragmentación transparente vs. no transparente

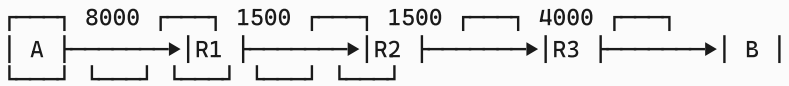
Este tema es de la **capa de red** pero va acá porque es el otro uso histórico del control de caudal (es el caso donde el router tiene que decidir qué hacer con un paquete que no entra). Es la **Pregunta 4 del Examen 2007-12** (10 pts).

```
EL PROBLEMA
──────────

Un mensaje (o paquete) de tamaño N tiene que atravesar una cadena de
enlaces con MTU distinta. Si N > MTU de algún salto, no pasa.

La pregunta es: ¿dónde y cómo lo divido?
```

(a) Fragmentación transparente




```
▲ ▲ ▲
│ │ └── el último router con MTU
│ │ 4000 REENSAMBLA antes de
│ │ entregarle a B
│ └── R2 fragmenta: ve que el paquete
│ de R1 no entra en su MTU
└── R1 fragmenta (porque el destino tiene MTU menor)

CARACTERÍSTICAS:
· Cada router FRAGMENTA por su cuenta.
· Cada fragmento lleva un NÚMERO DE FRAGMENTO mínimo
(número de secuencia), porque es un datagrama con posición.
· El receptor (o el ÚLTIMO router) REENSAMBLA.
· Los routers intermedios NO necesitan guardar estado.
```

Ventajas: - **Cero estado en los routers.** Cada router decide "por mí no pasa, fragmento", sin necesitar recordar qué estaba esperando. - **El receptor tiene el mensaje completo** (o al menos puede rearmarlo).

Desventajas: - **Overhead**: cada fragmento lleva un header (de capa 3 o de enlace) entero. Un mensaje de 8000 bytes que pasa por un enlace de MTU 1500 y que se fragmenta en cada salto puede acabar siendo **más caro que el mensaje original**. - **Se requiere buffering en el destino** para poder rearmar, y hay que saber cuándo recibir el último fragmento. - **Multiplicación de trabajo**: el mismo paquete se vuelve a fragmentar en cada salto donde no entra.

(b) Fragmentación no transparente

```
En la fragmentación NO transparente el router REENSAMBLA antes de
retransmitir al siguiente salto:

┌─┐  8000  ┌─┐  1500  ┌─┐  4000  ┌─┐
│A├────────►│R1├────────►│R2├────────►│B│
└─┘        └─┘        └─┘        └─┘
▲ ▲
│ └── R2 reensambla los fragmentos de R1
│ ANTES de pasarlo, porque él tiene un
│ MTU de 4000 hacia B
└── R1 fragmenta: le sale barato a él, porque es el que
puede hacerlo

CARACTERÍSTICAS:
· Cada router, ANTES de retransmitir, si recibió un fragmento, trata
de REENSAMBLARLO.
· El router decide fragmentar o no según la MTU DEL SALTO SIGUIENTE.
· Se puede usar un "número de fragmento" MUY pequeño (o ninguno).
```

Ventajas: - El **mensaje entero** siempre está disponible en cada salto. - Si el siguiente salto tiene MTU grande, el router reensambla y **ahorra fragmentar** y **reduce el total de transmisiones**.

Desventaja CRÍTICA: - **Cada router tiene que BUFFERIZAR para poder reensamblar**, o sea, necesita **memoria dedicada y estado** por fragmento. Eso es exactamente lo que MPLS vino a evitar, y es el motivo de la de MPLS y de los routers de alta capacidad.

La comparación que hay que saber de memoria

	TRANSPARENTE	NO TRANSPARENTE
¿Quién fragmenta?	el router de cada salto	un router, de a uno
¿Quién reensambla?	el destino final	el siguiente router
Estado en routers	NINGUNO	hay que bufferizar
Overhead	ALTO (n° de fragmento)	BAJO
Coste de memoria	baja	ALTA
Enlace de MTU grande después	Uniforme requiere el mismo	no hace falta

---

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Leaky bucket con ráfaga de entrada

Datos. `b = 100 bytes`, `r = 10 bytes/s`. Llegan paquetes de **20 bytes**. La tasa de entrada es de **5 paquetes cada 100 ms** (o sea 50 paquetes/s = 1000 B/s).

a) ¿Cuántos paquetes entran en 1 segundo?

Paso 1 — tiempo de vaciado. `f = b / r = 100 / 10 = 10 s`. O sea que la cubeta tarda **10 segundos** en vaciarse. Es lentísima comparada con el período de llegada.

Paso 2 — simular segundo a segundo.

```
contenido inicial = 0 B

t entrada ¿cabe? contenido goteo (10 B/s)
————— ———————— ———————— ———————— ——————————————————
0.00 20 B (1) 20 ≤ 100 → 20 B –
0.10 20 B (2) 40 ≤ 100 → 40 B –
0.20 20 B (3) 60 ≤ 100 → 60 B –
0.30 20 B (4) 80 ≤ 100 → 80 B –
0.40 20 B (5) 100 ≤ 100 → 100 B –
————— la cubeta está LLENA a los 0,40 s —————
0.50 20 B (6) 120 > 100 → DESCARTA
0.60 20 B (7) DESCARTA
... ... (todo lo que llega hasta que gotee)
```

Paso 3 — sale. El goteo es de 10 B/s, o sea **1 paquete cada 2 s** (`20 B / 10 B/s = 2 s por paquete`). En **1 segundo salen 0 paquetes** (o 1 si justo en el borde).

Respuesta: en 1 s entran 5 paquetes y salen 0,5. Se descartan ~45 paquetes.

b) ¿Cuántos paquetes se necesitan para vaciar la cubeta?

```
100 B / 20 B por paquete = 5 paquetes
y como el goteo entrega 1 paquete cada 2 s ⇒ 5 × 2 = 10 segundos
(que es f = b/r = 10 s ✓)
```

2.2 Ejercicio 2 — Token bucket con tokens acumulados

Datos. `b = 50 tokens`, `r = 10 tokens/s`, paquetes de **10 bytes** cada uno (1 token por byte).

a) Llegan 8 paquetes de golpe a `t = 0`. ¿Cuántos salen?

```
tokens iniciales = b = 50 (cubeta LLENA)

t=0, P1: tokens 50 ≥ 10 → tokens = 40, SALE
t=0, P2: tokens 40 ≥ 10 → tokens = 30, SALE
t=0, P3: tokens 30 ≥ 10 → tokens = 20, SALE
t=0, P4: tokens 20 ≥ 10 → tokens = 10, SALE
t=0, P5: tokens 10 ≥ 10 → tokens = 0, SALE ← los 5 SALEN SEGUIDOS
t=0, P6: tokens 0 < 10 → ESPERA = ráfaga de 50 bytes
t=0, P7: espera
t=0, P8: espera

⇦ CONTRASTE CON LEAKY: los mismos 5 paquetes, la misma cubeta de 50,
con b=100 y r=10 el leaky los saca de a uno, cada 1 s. El token
los saca SEGUIDOS. ESO es la ráfaga.
```

b) ¿Cuándo sale P6?

```
Los tokens se generan a 10/s: 1 token cada 0,1 s.
Necesita 10 tokens ⇒ 10 × 0,1 s = 1,0 s.
P6 sale a t = 1,0 s (y P7 a 1,1 s, P8 a 1,2 s).
```

c) Con los mismos datos, un leaky bucket con `b=50, r=10`. ¿Cuándo sale P5?

```
f = b/r = 50/10 = 5 s para vaciar la cubeta.
Sale 1 paquete cada 10 B / 10 B/s = 1 s.
P1 a t=1,0 s ; P2 a 2,0 ; P3 a 3,0 ; P4 a 4,0 ; P5 a 5,0.
NUNCA dos seguidos. ← la diferencia de fondo
```

2.3 Ejercicio 3 — Qué DSCP le pongo a tres clases de tráfico

Datos. Una red de empresa con tres clases de tráfico: (1) **voz** sobre RTP; (2) **descarga de archivos** por FTP; (3) **backup** de disco de 3 TB por la noche.

```
╔═══════════╦═══════════════════════════════════════════════════════════════╗
║ CLASE ║ DSCP ║ POR QUÉ ║
╠═══════════╬═══════════════════════════════════════════════════════════════╣
║ 1. VOZ ║ EF (DSCP = 46 = 101110b) ║ No puede esperar. Ni ║
║ ║ ║ jitter ni retardo ║
║ ║ ║ tolerables. Se marca con ║
║ ║ ║ la MÁXIMA prioridad de ║
║ ║ ║ descarte: si hay ║
║ ║ ║ congestión, se tira lo ║
║ ║ ║ que sea antes que la voz.║
╠═══════════╬═══════════════════════════════════════════════════════════════╣
║ 2. FTP ║ AF12 (DSCP = 12 = 001100b) ║ Interactivo, sensible a ║
║ descarga ║ ║ el retardo (el usuario ║
║ ║ ║ espera el archivo). ║
║ ║ ║ Preferencia 2: se tira ║
║ ║ ║ después que las pref. 1 ║
║ ║ ║ de la misma clase. ║
╠═══════════╬═══════════════════════════════════════════════════════════════╣
║ 3. BACKUP ║ AF41 (DSCP = 34 = 100010b) ║ BULK: no importa nada ║
║ de 3 TB ║ ║ si tarda horas. Se ║
║ ║ ║ descarta de las primeras ║
║ ║ ║ (preferencia 1, la más ║
║ ║ ║ baja). Con policing en ║
║ ║ ║ el edge a 1 Mbps, el ║
║ ║ ║ resto se descarta o se ║
║ ║ ║ remarkea. ║
╚═══════════╩═══════════════════════════════════════════════════════════════╝
```

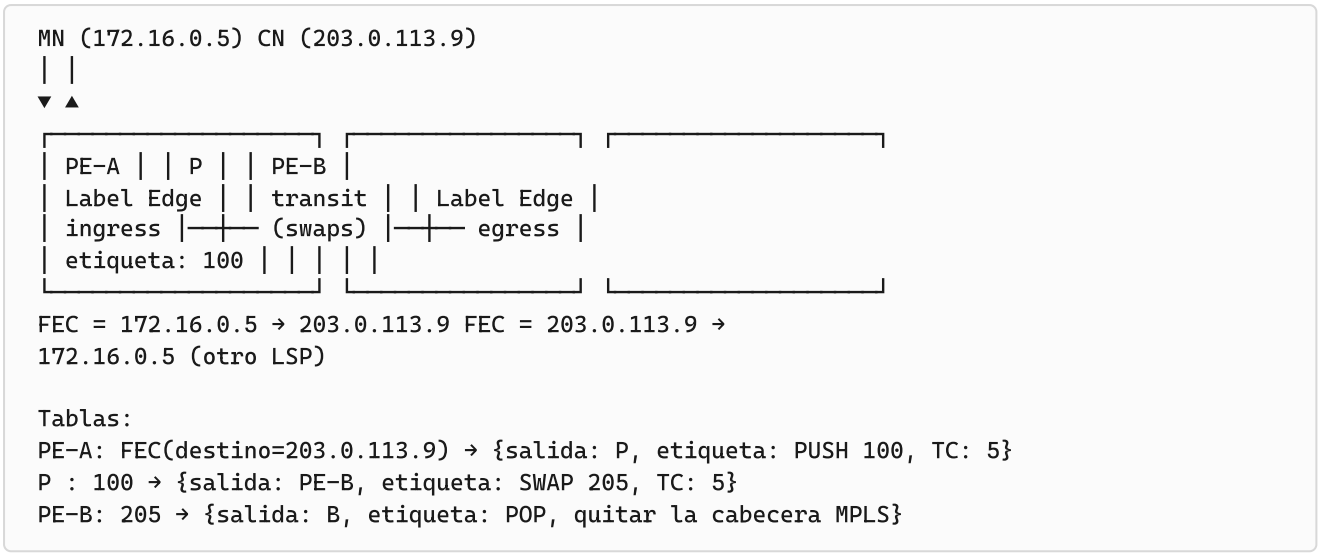
¿Y por qué el backup NO va con DSCP 0 (Best Effort)?

```
Porque el backup, aunque sea bulk, va a COMPETIR con la web del
usuario por el mismo enlace de salida. Con DSCP 0, no tenés
ninguna prioridad garantizada. Con AF41 + policing en el borde,
el backup:
· tiene su propia categoría en la cola
· está explícitamente en la última posición de descarte
· si sobra banda, la usa (el token bucket le deja ráfagas)
· si falta banda, es el primero que se sacrifica

Ése es el DIFERENCIAL de DiffServ: no es "más rápido" para el
backup, es "no molestar al resto".
```

2.4 Ejercicio 4 — El flujo de un paquete por un LSP

Datos. Topología con tres routers MPLS:



El recorrido, con el cambio de etiqueta en cada router:

```
Paso 1 — MN envía el paquete IP
cabecera IP: src=172.16.0.5 dst=203.0.113.9 (etiqueta: NO HAY)

Paso 2 — PE-A (INGRESO, hace el PUSH)
· Mira la dirección IP de destino (203.0.113.9) para clasificar el
flujo en la FEC correspondiente
· Consulta su tabla: FEC → {salida: P, etiqueta: 100, TC: 5}
· EMPUJA la etiqueta 100 entre la cabecera IP y el payload
· Copia el TC=5 (que es el DSCP/prioridad) a la etiqueta

Ahora en el cable:
[IP: src=172.16.0.5 dst=203.0.113.9] [Label 100 | TC 5] [datos]

Paso 3 — P (TRÁNSITO, hace el SWAP)
· Mira SÓLO los 20 bits de la etiqueta: 100
· Búsqueda EXACTA: 100 → {salida: PE-B, etiqueta: 205}
· Cambia 100 por 205. NO toca la dirección IP. NO consulta longest
prefix match. NO sabe a dónde va el paquete realmente: no importa.

[IP: src=172.16.0.5 dst=203.0.113.9] [Label 205 | TC 5] [datos]
▲ ▲
nunca la mira cambió 100 → 205

Paso 4 — PE-B (EGRESO, hace el POP)
· Mira SÓLO la etiqueta: 205
· 205 → {acción: POP}
· QUITA la etiqueta MPLS entera

[IP: src=172.16.0.5 dst=203.0.113.9] [datos]
▲
ahora sí es un paquete IP normal

Paso 5 — B recibe el paquete IP y cre que nunca vio MPLS.

RESUMEN DE ETIQUETAS EN EL CABLE:
┌────────┬──────────┬─────────┬──────────┬──────┐
│ MN │ PE-A │ P │ PE-B │ B │
├────────┼──────────┼─────────┼──────────┼──────┤
│ (nada) │ 100 │ 205 │ (pop) │(nada)│
│ IP │ IP │ IP │ IP │ IP │
└────────┴──────────┴─────────┴──────────┴──────┘
```

Interpretación — y esto es lo que hay que escribir: la etiqueta **cambia en cada salto**, pero **sólo cambia la etiqueta**. La cabecera IP viaja intacta de extremo a extremo sin que ningún router intermedio la mire. El único momento en que alguien mira la IP es el **ingress** (para clasificar) y el **egress** (que la necesita para entregarla). En el medio, **nadie sabe para dónde va el paquete**, y eso es exactamente lo que hace que MPLS sea rápido: la decisión se toma con 20 bits de búsqueda exacta en lugar de 32 bits de longest prefix match.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 Examen 2007-12, Pregunta 1 (10 puntos) — Leaky bucket y token bucket

> **Fuente:** Examen `eirc0712` / `sirc0712` — 27 de diciembre de 2007.
>
> **Enunciado:** Explique el algoritmo de **cubeta con goteo** (*Leaky Bucket*) y **cubeta con tokens** (*Token Bucket*).

(El enunciado no trae números: es puramente conceptual. Eso es lo que hay que resolver.)

La respuesta estructurada, que es la que hay que aprender

Leaky bucket (cubeta con goteo) — modelo:

```
Un paquete llega ⟶ [ CUBETA de capacidad b ] ⟶ sale a r bytes/s
▲ ▲
| |
entrada a ritmo el AGUJERO
ARBITRARIO gotea SIEMPRE
al mismo ritmo

· Si la cubeta tiene lugar (contenido + L ≤ b): el paquete ENTRA.
· Si NO tiene lugar: el paquete se DESCARTA (o se marca para otro
tratamiento: es la base del POLICING).
· El goteo NO SE DETIENE aunque la cubeta esté vacía: el paquete que
está en el fondo sale igual.

CONSECUENCIA: la salida tiene ritmo CONSTANTE. Es imposible que salgan
dos paquetes juntos.
```

Token bucket (cubeta con tokens) — modelo:

```
Un token se genera ⟶ [ CUBETA de capacidad b tokens ]
cada 1/r segundos ▲ |
(hasta b, no más) | ▼
un paquete de L bytes si hay ≥ L tokens:
CONSUME L tokens SALE
si no hay: ESPERA

· Los tokens se GENERAN a ritmo constante r (o 1 token por "unidad de
caudal"), con tope en b.
· Para enviar un paquete de L bytes hacen falta L tokens.
· Si no hay tokens: el paquete ESPERA (no se descarta).

CONSECUENCIA: la salida puede tener RÁFAGAS de hasta b bytes.
```

El cuadro comparativo, que es lo que el corrector quiere ver:

	LEAKY BUCKET	TOKEN BUCKET
Concepto	agua en una cubeta	fichas en una cubeta
Qué se genera	NADA	r fichas/s
Si no hay crédito	DESCARTA el paquete	el paquete ESPERA
Ritmo de salida	SIEMPRE constante	con ráfagas de hasta b
Ráfaga	imposible	posible
Usa toda la capacidad	no (se desperdicia si	sí
si sobra	la cubeta está vacía)	
Retrasa	sí (inherentemente,	sí (si no hay tokens)
	por el buffer)	
Uso típico	control de caudal	control de caudal
	"duro" / exacto	"blando" / con holgura

La frase de cierre que hay que escribir (es la que más puntos da):

> *"El leaky bucket produce un flujo de salida constante a costa de descartar todo lo que exceda; el token bucket permite ráfagas de hasta `b` bytes a costa de retardar los paquetes que no tienen tokens. Ambos limitan el caudal medio a `r`, pero el token bucket es el que además absorbe las ráfagas de entrada sin perder información."*

Y la justificación de por qué conviene cada uno (ojo: el enunciado no lo pide, pero si lo agregás suma):

- El **leaky bucket** sirve cuando **no se puede perder nada** (por ejemplo, un enlace de control donde cada paquete importa y hay que mandarlos a un ritmo constante).
- El **token bucket** sirve cuando la fuente tiene **ráfagas legítimas** que no querés perder (por ejemplo, el tráfico de una aplicación por ráfagas).

3.2 Examen 2009-03, Pregunta 4 (10 puntos) — DiffServ: edge, core y campo de marcado

> **Fuente:** Examen `eirc0903` / `sirc0903` — 20 de febrero de 2009.
>
> **Enunciado:** Considere la arquitectura de Servicios Diferenciados. **a)** Describa las funciones que debe realizar un "**edge router**" y un "**core router**", y qué campo de IPv4 se utiliza para marcar los tipos de servicio. **b)** Describa el mecanismo de forwarding "**Per Hop Behaviour**".

Solución oficial (textual):

> **a)** El "edge router" debe clasificar y marcar el tráfico que ingresa a la red. El "core router" debe encaminar el tráfico de acuerdo a las marcas definidas en el ingreso, y al PHB definido para cada clase de tráfico, usando el **campo TOS de IPv4**.
>
> **b)** El PHB son las políticas de tratamiento del tráfico de acuerdo a las marcas realizadas en el borde de la red. Por ejemplo prioridades, % del ancho de banda, etc.

La resolución completa

a) Las dos funciones y el campo.

```
EDGE ROUTER (router de borde) — hace TRES cosas:
─────────────────────────────────────────────────────────────────────────

1. CLASIFICAR: determinar a qué CLASE de servicio pertenece el paquete.
Usa un clasificador: mirando (puerto, protocolo, dirección IP, longitud,
contenido). Típicamente es una tabla:
(IP origen, protocolo, puerto) ⟶ clase
Ejemplo: (RTP, puerto 16384-32767) → EF (voz)
(HTTP, puerto 80) → AF11 (web)
(todo lo demás) → 0 (best effort)

2. MARCAR: escribir el DSCP en el campo DS (byte TOS) del paquete.
Es el ÚNICO router que hace esto. A partir de ahí, la marca viaja sola
por la red.

3. (a veces) SHAPER o POLICER: entregar la clase con el caudal que la
política de esa clase permite.

─────────────────────────────────────────────────────────────────────────
CORE ROUTER (router del núcleo) — hace UNA sola cosa:
─────────────────────────────────────────────────────────────────────────
ENCAMINAR aplicando el PHB definido para la marca que viene en el paquete.

· Mira el byte DS.
· Mira su tabla de política: DSCP → {cola, prioridad de descarte,
ancho de banda mínimo, discard}
· Pone el paquete en la cola que corresponde, con la prioridad que
corresponde, y reenvía.
· NO clasifica. NO re-marca. NO sabe qué flujo es. NO guarda estado por
flujo. NO hay señalización.

─────────────────────────────────────────────────────────────────────────
EL CAMPO DE IPv4: el byte TOS (Type of Service) — que HOY se llama
─────────────────────────────────────────────────────────────────────────
el campo DS (Differentiated Services).
```

El layout del byte (esto hay que dibujarlo):

```
Byte TOS/DS de la cabecera IPv4 (offset 1), 8 bits:

┌──────────────────────────────┬──────────┐
│ DSCP (6 bits) | ECN(2) |
├──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┤
| b5 | b4 | b3 | b2 | b1 |b0 | ECN | ECN |
└──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┘
▲ ▲
└── 6 bits de DSCP: └── 2 bits de ECN (NO-ECN / AWARE / CE)

⚠️ El orden importa: DSCP primero, ECN después. El error típico es
invertirlos porque el layout "histórico" del byte TOS tenía la
precedencia (3 bits) al principio.
```

El nombre de los PHB que hay que saber (nombres + para qué):

PHB	DSCP	Significado
Default / BE	0	sin tratamiento especial
EF (*Expedited Forwarding*)	46	voz: mínima probabilidad de descarte y **no se encola detrás de nada**
AF11..AF43	10..38	4 clases (garantía de bw) × 3 drop precedences (qué se tira primero)
CS (*Class Selector*)	1xxxxxx	heritage de los valores viejos del byte TOS

b) El PHB (Per Hop Behaviour).

Definición precisa (es la que hay que escribir):

> **El PHB es el conjunto de políticas y reglas de tratamiento que un router aplica a los paquetes que llevan una determinada marca (un determinado DSCP), en CADA salto.** Es la que se define "salto a salto", sin que ningún router necesite saber nada del flujo, ni del camino, ni del destino final.

Lo que un PHB puede decidir (y estos son los ejemplos que da la solución oficial):

```
· A qué COLA va el paquete (prioridad entre colas)
· Qué PORCENTAJE del ancho de banda tiene garantizado cada clase
· Qué PRIORIDAD DE DESCARTE tiene: a quién se tira primero cuando
hay congestión
· Si puede usar el REBAJO DE ETIQUETA (no encolarse)
· Qué POLICING/SHAPING se le aplica

Ejemplo concreto con tres clases:

┌──────────┬──────────────┬──────────┬──────────────────────┐
│ Clase │ bw mínima │ Cola │ Descarte │
├──────────┼──────────────┼──────────┼──────────────────────┤
│ EF (voz) │ 2 Mbps │ 1 (la 1ª) │ NUNCA se descarta │
│ AF11(web) │ 4 Mbps │ 2 │ después de las AF │
│ Default │ el resto │ 3 │ se descarta primero│
└──────────┴──────────────┴──────────┴──────────────────────┘

Eso es el PHB de AF11: "garantizame al menos 4 Mbps, mandame por la
cola 2, y si tengo que tirar algo, tirá lo de la cola 3 antes que
lo mío".
```

El contraste que hay que hacer al final (es lo que el enunciado busca):

```
"Per Hop" significa SALTO A SALTO:
· el router NO conoce el camino completo
· el router NO mantiene estado por flujo
· el router NO tiene quearse con los demás (no hay señalización)
· la política se aplica IGUAL en cada salto

⇒ por eso escala. Un router de backbone con DiffServ tiene un
clasificador con, digamos, 12 clases. No 1 millón de flujos.
```

> Tope para 10 puntos: (a) los tres pasos del **edge** (clasificar, marcar, y opcionalmente shaper/policer), la **una** función del **core** (encaminar aplicando el PHB de la marca), y el **nombre exacto del campo** (byte

TOS / DS) con el layout de 6 + 2 bits. (b) la definición de PHB, los ejemplos de la solución oficial (prioridad, % de ancho de banda) y el contraste de por qué escala.

3.3 Examen 2010-07, Pregunta 4 (8 puntos) — El paradigma de DiffServ

> **Fuente:** Examen `eirc1007` / `sirc1007` — 27 de julio de 2010.
>
> **Enunciado:** a) Explique el **paradigma de funcionamiento** de la arquitectura DiffServ. b) Mencione las **funciones que se encuentran en el edge** y las que se realizan en el **core** de dicha arquitectura.

Solución oficial: "*Diapos 83, 84 y 86 de Clase 2 de Multimedia*" — es decir, **no hay texto de solución publicada**: la cátedra se remitió a las transparencias del curso. Eso significa que **la respuesta esperada es la del teórico**, y hay que darla completa y bien estructurada. *(solución propia, reconstruida del material del curso)*

La resolución

a) El paradigma de DiffServ, en tres pasos.

```
EL PARADIGMA, en una frase:
"En lugar de reservar recursos para cada flujo (IntServ), se CLASIFICA
el tráfico en el borde en un número PEQUEÑO de clases, se MARCA con un
código de 6 bits que viaja en la cabecera IP, y cada router del núcleo
aplica un COMPORTAMIENTO POR SALTO (PHB) según esa marca, SIN
señalización y SIN guardar estado por flujo."
```

Los tres pasos con su nombre técnico (esto es lo que hay que escribir):

Paso	Nombre técnico	Quién lo hace	En qué consiste
1	**Classification** (clasificación)	el **edge router**	Mira el paquete (5-tupla, puertos, protocolo, longitud, contenido) y decide a qué **clase de servicio** pertenece, según una **tabla de clasificación**
2	**Marking** (marcado)	el **edge router**	Escribe el **código de 6 bits (DSCP)** en el **campo DS** del byte TOS de la cabecera IP
3	**PHB** (*Per Hop Behaviour*)	**cada** router del **core**	Aplica el comportamiento definido para ese DSCP: cola, prioridad de descarte, ancho de banda mínimo, policing/shaping

Las TRES propiedades que hacen que DiffServ escale (esto es lo que el enunciado realmente pregunta, porque pregunta por el "paradigma"):

```
┌──────────────────────────────────────────────────────────────────┐
│ 1. SIN SEÑALIZACIÓN │
│ No hay RSVP, no hay handshake, no hay setup. │
│ El router del edge NO pregunta a nadie "podés reservarme │
│ 2 Mbps?". Simplemente marca el paquete y lo manda. │
├──────────────────────────────────────────────────────────────────┤
│ 2. SIN ESTADO POR FLUJO │
│ El core NO guarda información por conexión. │
│ Guarda UNA TABLA PEQUEÑA: DSCP → comportamiento. │
│ Con 6 bits de DSCP hay, como máximo, 64 valores │
│ (y en la práctica se usan 12 o menos). │
├──────────────────────────────────────────────────────────────────┤
│ 3. CLASES, NO GARANTÍAS INDIVIDUALES │
│ La garantía es "la clase AF11 tiene 4 Mbps garantizados", │
│ no "este flujo tiene 4 Mbps garantizados". │
│ Si dos flujos están en AF11, se reparten los 4 Mbps. │
│ El enlace es COMPARTIDO, no reservado. │
└──────────────────────────────────────────────────────────────────┘
```

El contraste con IntServ, que hay que incluir porque es lo que el enunciado quiere que se entienda:

```
┌──────────────┬──────────────────────┬──────────────────────┐
│ │ IntServ │ DiffServ │
├──────────────┼──────────────────────┼──────────────────────┤
│ Granularidad│ POR FLUJO │ POR CLASE │
│ Señalización│ SÍ (RSVP) │ NO │
│ Estado │ POR FLUJO, en │ POR CLASE (12-ish) │
│ │ cada router │ en cada router │
│ Guarantee │ ABSOLUTA │ RELATIVA (dentro de │
│ │ (bw, retardo, │ la clase) │
│ │ jitter, pérdida) │ │
│ Escala │ NO │ SÍ │
│ Estado hoy │ ≈ no se despliega │ EL ESTÁNDAR │
└──────────────┴──────────────────────┴──────────────────────┘
```

b) Las funciones del edge y del core, en columnas.

```
┌──────────────────────────── EDGE ROUTER ────────────────────────────┐
│                                                                     │
│ · CLASIFICAR el tráfico (mirar 5-tupla, puertos, protocolo)         │
│ · MARCAR con el DSCP en el campo DS del byte TOS                    │
│ · (opcional) SHAPER para limitar el caudal de cada clase            │
│ · (opcional) POLICER lo que excede de un subscribed rate            │
│ · (opcional) RE-MARKING: si un paquete viene de adentro con         │
│ un DSCP que el cliente NO tiene derecho a usar, bajarlo             │
│ (ej: un usuario que pone EF en su VoIP hogareña)                    │
│                                                                     │
│ ⚠️ Es el ÚNICO lugar donde se toca la marca.                        │
└─────────────────────────────────────────────────────────────────────┘

┌──────────────────────────── CORE ROUTER ────────────────────────────┐
│                                                                     │
│ · LEER el DSCP del campo DS                                         │
│ · ENCAMINAR el paquete hacia el siguiente salto                     │
│ · APLICAR el PHB de ese DSCP:                                       │
│ - elegir la COLA de salida                                          │
│ - respetar el ancho de banda mínimo de la clase                     │
│ - respetar la prioridad de descarte de la clase                     │
│ - aplicar policing/shaping local si el admin lo configuró           │
│                                                                     │
│ ⚠️ NO clasifica. NO marca. NO re-marca. NO señaliza.                │
│ NO guarda estado por flujo.                                         │
└─────────────────────────────────────────────────────────────────────┘
```

El detalle fino que hace que DiffServ funcione, y que hay que decir:

> **"El core no necesita confiar en el edge."** O más precisamente: **la marca viaja dentro del paquete**, no en el estado de los routers. Eso significa que **cualquier router de la ruta puede leerla sin haber={""} nada con nadie. Y por eso el modelo es** extremadamente escalable**: el estado de un router depende del** número de clases**, que es constante, y** no** del número de flujos, que crece sin límite.

> Tope para 8 puntos: los tres pasos con su nombre (classification, marking, PHB), quién los hace (edge / edge / todos), las **tres propiedades** que lo hacen escalar (sin señalización, sin estado por flujo, clases en lugar de flujos), y la lista de funciones del edge y del core.

3.4 Examen 2007-12, Pregunta 4 (10 puntos) — Fragmentación transparente y no transparente

> **Fuente:** Examen `eirc0712` / `sirc0712` — 27 de diciembre de 2007. *(El PDF de este examen es un escaneo sin texto en el apartado de soluciones; el enunciado sí se lee. La resolución es propia, reconstruida del material del curso.)*
>
> **Enunciado:** Explique en qué consiste la **fragmentación transparente** y la **fragmentación no transparente**.

La resolución

El problema, primero (para ubicarse):

```
Un mensaje (o paquete) de tamaño N > MTU de algún salto. La MTU
(Maximum Transfer Unit) es el tamaño máximo de PDU de capa 3 que un
enlace transmite sin fragmentar.

┌─┐  8000  ┌─┐  1500  ┌─┐  1500  ┌─┐  4000  ┌─┐
│ A ├──────►│R1├──────►│R2├──────►│R3├──────►│ B │
└─┘        └─┘        └─┘        └─┘        └─┘

El mensaje tiene que pasar por enlaces de 8000, 1500, 1500 y 4000 bytes.
Los de 1500 no lo dejan pasar entero. La pregunta es: ¿QUIÉN lo parte y
CUÁNDO?
```

(a) Fragmentación transparente

```
Los routers INTERNOS (los de tránsito) NO se enteran de que hay
fragmentación. Cada router fragmenta lo que no le entra por el enlace de
SALIDA, y el DESTINO FINAL (o el último router) reensambla.

┌─┐  8000  ┌─┐  1500  ┌─┐  1500  ┌─┐  4000  ┌─┐
│ A ├──────►│R1├──────►│R2├──────►│R3├──────►│ B │
└─┘        └─┘        └─┘        └─┘        └─┘
✓ ▲ FRAG. ▲ FRAG. ▲ REENS. ▲
entero el 1ro el 2do aquí recibe
(N < 8000) que no que no (antes de el MENSAJE
entra entra entregar) ENTERO

· R1 ve que su salida es 1500 y su entrada le trajo 8000: fragmenta.
Cada fragmento lleva un NÚMERO DE FRAGMENTO (mínimo) en el header.
· R2 recibe un fragmento de 1500 que NO entra en su salida (1500): o
lo deja pasar (entra justo) o, si el mensaje era más grande, lo
vuelve a fragmentar.
· El ÚLTIMO router con MTU distinto (o el destino) REENSAMBLA: junta
los fragmentos por número, en el orden correcto, y reconstituye el
mensaje.
```

Características: - **Cada router fragmenta**, según su propia MTU de salida. Los routers no "coordinan": cada uno hace lo suyo. - **Los fragmentos llevan número** (o offset, en IPv4: `Fragment Offset`), porque son paquetes **independientes** que pueden perderse. - **El reensamblaje ocurre al final** (en el destino o en el último router). - **Los routers intermedios NO necesitan estado**: no saben de antemano que van a recibir fragmentos, no guardan nada. Cada uno ve un paquete más o menos grande y decide.

Ventajas: - **CERO estado en los routers intermedios.** Es la ventaja grande, y es la razón por la que existió: los routers de la época no tenían memoria para guardar reensamblajes parciales. - El **mensaje entero** le llega al destino. - Es **simple** de implementar.

Desventajas: - **Overhead grande**: cada fragmento lleva un header completo. Un mensaje de 8000 bytes que pasa por tres saltos de MTU 1500 puede terminar siendo, en el cable, **más caro que el mensaje original**: 6 fragmentos × 1500 bytes = 9000 bytes. - **Requiere buffering en el destino** (o en el último router) y saber cuándo llegó el último fragmento. - **Trabajo repetido**: el mismo mensaje se fragmenta y

reensambla en cada salto distinto. - Si se pierde UN fragmento, se pierde el mensaje entero (hay que retransmitir todo).

(b) Fragmentación no transparente

```
En la fragmentación NO transparente, CADA ROUTER REENSAMBLA el mensaje
ANTES de retransmitirlo al siguiente salto. La fragmentación es
"opaca" para el siguiente salto.

┌─┐  8000  ┌─┐  1500  ┌─┐  4000  ┌─┐
│A├──────►│R1├──────►│R2├──────►│B│
└─┘        └─┘        └─┘        └─┘
✓ ▲ FRAG. ▲ REENS. ▲
entero + REENS. + pasa recibe
(antes de ENTERO el mensaje
fragmentar)

· R1: le llega el mensaje entero de 8000. Su salida es 1500.
Decide: ¿le conviene más fragmentar acá y mandar 1500, o
reensamblar primero? Fragmenta y manda 6 fragmentos de 1500.
· R2: RECIBE los fragmentos, REENSAMBLA (los junta en el mensaje de
8000), y ANTES de pasarlo a B (que tiene MTU 4000) mira: entra
entero. Lo pasa entero. NO lo vuelve a fragmentar.

El punto: CADA router sabe si su salida aguanta el mensaje entero, y
decide en consecuencia.
```

Características: - El router **reensambla antes de retransmitir** al siguiente salto. - La decisión de fragmentar se toma **según la MTU del salto siguiente**, no según el mensaje que llegó. - Cada fragmento puede llevar **un número de fragmento mínimo** (o incluso nada), porque el reensamblaje ocurre en el router que fragmentó.

Ventajas: - **El mensaje entero siempre está disponible en cada salto** (o por lo menos en cada router que lo recibió fragmentado). - Si el siguiente salto tiene MTU grande, el router reensambla y **no vuelve a fragmentar**: se **ahorra transmisiones**. - **Overhead bajo**: si el mensaje se reensambla pronto, hay menos fragmentos circulando.

Desventajas CRÍTICA: - **Cada router que recibe fragmentos tiene que BUFFERIZAR** para poder reensamblar. O sea, **memoria dedicada por fragmento recibido**. Ése es el costo que **MPLS vino precisamente a eliminar**, y la razón por la que el hardware de alta capacidad hizo que los MPLS switches reemplazaran a los routers IP traditional. - Un mensaje que cruza muchos routers con MTU distinto **se reensambla y re-fragmenta muchas veces**, que es trabajo inútil.

La comparación, que es lo que hay que escribir al final

	TRANSPARENTE	NO TRANSPARENTE
¿Quién fragmenta?	el router, salto a salto	un router, y luego otro
¿Quién reensambla?	el DESTINO (o el último router)	el siguiente router (antes de retransmitir)
Estado en routers	NINGUNO	hay que BUFFERIZAR
Overhead	ALTO (nº de fragmento por cada salto)	BAJO (se reensambla pronto)
Coste de memoria	BAJA	ALTA
¿Cuándo decide?	cuando un paquete NO entra en su salida	cuando va a RETRANSMITIR al siguiente salto
MTU del camino	irrelevante (o casi)	importa mucho
Ejemplo típico	el "mini-packet" de la ATM ("virtual channels")	el tráfico entre routers con MTU distinto

Y el corolario histórico que es lo más importante de esta pregunta:

> La fragmentación **no transparente** es **precursora de MPLS**. MPLS es esencialmente la misma idea (la capa de red queda "opaca" para los routers intermedios) pero **sin fragmentar ni reensamblar nunca**: los routers MPLS reenvían por **etiqueta**, sin mirar la longitud del paquete, y sin necesidad de reensamblar. Por eso MPLS es **más eficiente que las dos**.

> Tope para 10 puntos: la definición de cada una, quién fragmenta y quién reensambla en cada caso, la **ventaja clave** de la transparente (cero estado) y la **desventaja clave** de la no transparente (buffering), y el corolario histórico con MPLS.

3.5 Examen 2007-12, Pregunta 5 (5 puntos) — MPLS: router de ingreso y de egreso

> **Fuente:** Examen `eirc0712` / `sirc0712` — 27 de diciembre de 2007.
>
> **Enunciado:** ¿Qué función cumplen el **router de ingreso** y el **egreso** en MPLS?

(La solución de este examen viene escaneada sin texto para esta pregunta; la resolución es la del material del curso, tal como la define `sirc0608 P5`, que sí está en texto. Ver §3.6.)

La respuesta

Router de INGRESO (*ingress*, también LER — *label edge router*):

```
· RECIBE los paquetes de la capa de red (IP, ATM, Ethernet…) que entran
al dominio MPLS por este router.
· HACE FORWARDING NORMAL: mira la dirección IP de destino, hace el
longest prefix match, y determina el siguiente salto.
· DEDUCE LA FEC (Forwarding Equivalence Class): agrupa los paquetes que
van a ser tratados igual. La FEC no es un paquete: es un
CONJUNTO de paquetes. Ejemplo: "todo lo que va del prefijo A al
prefijo B".
· ASIGNA LA ETIQUETA: consulta su tabla de LABEL INBINDING (que dice
"para la FEC F, la etiqueta que corresponde es X") y empuja la
etiqueta X (acción PUSH).
· (opcional) IMPONE la QoS,copiando el DSCP al campo TC de la etiqueta.

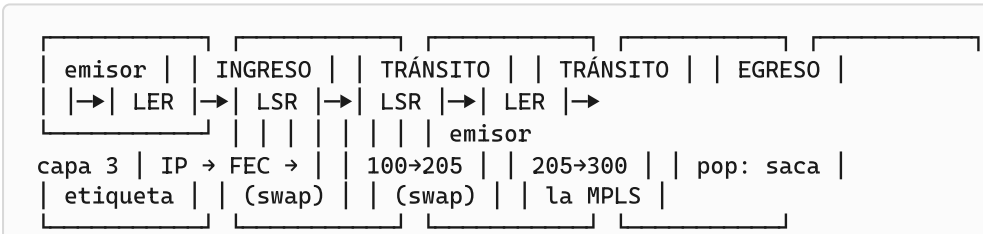
Es el ÚNICO router del LSP que mira la dirección IP de capa 3.
```

Router de EGRESO (*egress*, también LER):

```
· RECIBE los paquetes MPLS que llegan por el LSP.
· SACA la etiqueta MPLS (acción POP): quita la cabecera de etiqueta
completa.
· PONE la CABECERA DE CAPA 3 de abajo (la que el paquete venía
encapsulando).
· Entrega el paquete al siguiente router (o a la aplicación local).

Es el otro ÚNICO router que puede necesitar mirar la IP de capa 3.
```

Y el resumen del LSP (lo que hay que dibujar):



> **Tope para 5 puntos:** los **cinco verbos** del ingreso (**recibe, hace forwarding IP, deduce la FEC, asigna la etiqueta, la empuja**) y los **dos** del egreso (**saca la etiqueta, pone la cabecera de capa 3**), más el hecho de que **ninguno de los dos es un router de tránsito**.

3.6 Examen 2006-08, Pregunta 5 (5 puntos) — MPLS: etiqueta, LSP y roles

> **Fuente:** Examen `eirc0608` / `sirc0608` — 5 de agosto de 2006.
>
> **Enunciado:** Describa los conceptos fundamentales de MPLS: la **etiqueta**, el **LSP** y los **diferentes roles** que cumplen los routers en el modelo MPLS.

Solución oficial (textual):

> **Etiqueta:** - **Label**: identificador de flujo, índice en tabla - **Experimental bits** (QoS): clase de servicio - **BS**: stacking de etiquetas - **TTL**: usado de la misma forma que en IP
>
> **Label Switched Path (LSP):** túnel unidireccional (simplex) a través de la red, formado por saltos controlados por las etiquetas.
>
> **Label Switch Router:** - **Ingress LSR** ("head-end LSR"): examina los paquetes IP entrantes y los clasifica asignándolos a una FEC; genera el encabezado MPLS y asigna la etiqueta inicial - **Transit LSR**: reenvía los paquetes MPLS usando la etiqueta y haciendo swapping - **Egress LSR** ("tail-end LSR"): remueve el encabezado MPLS

La resolución: por qué cada campo de la etiqueta existe

```
La etiqueta MPLS NO es una dirección. Es un ÍNDICE EN UNA TABLA.
┌──────────────────────────────┬──────────┬──────────┬──────┐
│ Label (20 bits) │ TC (3) │ S (1) │ TTL │
├──────────────────────────────┼──────────┼──────────┼──────┤
│ ¿POR QUÉ EXISTE? │ ¿QoS? │ ¿anidado?│ ¿loop?│
│ 20 bits = 1.048.576 flujos │ 8 niveles│ S=1 → hay│ mismo │
│ simultáneos POR ROUTER. │ de prio- │ etiqueta │ que en │
│ Suficiente para un router. │ ridad de │ debajo │ IP │
└──────────────────────────────┴──────────┴──────────┴──────┘
```

Explicación de cada uno (esto es lo que hay que developed):

1. **Label (20 bits).** Es un **número inventado por el router**, sin significado global. Cada router tiene su **propio** espacio de etiquetas y su **propia** tabla. Si el router de tránsito ve "etiqueta 100", no sabe nada del destino: sólo sabe "la 100 en MI tabla dice: mandá por la interfaz 3 con la etiqueta 205". Por eso 20 bits alcanzan: son 1.048.576 valores, y un router de tránsito de backbone maneja del orden de cientos de miles. **El 20 no es arbitrario**: tiene que ser **único dentro de ese router** y tener un **bit de reserva** (20 = 16 bits de espacio + 4 bits para usos especiales como el *IPv4 Explicit NULL*).
2. **TC / Experimental bits (3 bits).** Es el **lugar donde vive el QoS de MPLS**. Hay tres nombres posibles para el mismo campo según el RFC que se use (`EXP` — experimental, `TC` — traffic class, `Precedence`): 8 niveles de prioridad de descarte. Es el **análogo exacto** del DSCP, pero dentro del shim MPLS, no dentro de la cabecera IP.
3. **BS (Bottom of Stack) / S.** Es un bit que dice "**hay otra etiqueta debajo de esta**". Sirve para el **LSP tunnel anidado** (*label stack*): cuando un LSP atraviesa la red de otro operador, se mete **una segunda etiqueta encima** (como una hoja de papel sobre otra). El SP bit le dice al router "cuándo: si S=0, esta es la última etiqueta; si S=1, hay más debajo y tenés que mirar la siguiente".
4. **TTL (8 bits).** Se usa **exactamente igual que en IP**: se decrementa en cada salto MPLS. Sirve para (a) **prevenir loops** y (b) que `traceroute` funcione a través de MPLS. La diferencia con el TTL de IP: el TTL de MPLS cuenta **saltos MPLS**, no todos los saltos de la ruta (puede haber saltos de IP por debajo de la pila de etiquetas que no se cuentan).

El LSP (Label Switched Path):

> *"Un túnel unidireccional (simplex) a través de la red, formado por saltos controlados por las etiquetas."*

Desarrollado:

```
Un LSP es:
· UNIDIRECCIONAL (simplex). Para una comunicación bidireccional
hacen falta DOS LSP, uno en cada sentido.
· PREDEFINIDO (con la entrada/salida fija), a diferencia del
reenvío por IP que es "best effort" y puede cambiar de camino
en cada momento.
· Compuesto de saltos donde CADA SALTANDO se cambia la etiqueta
(swap), siguiendo la "tabla de reenvío por etiqueta".
· ⚠️ NO RESERVA RECURSOS. Esto es lo que más se confunde: un LSP
es un CAMINO, no un CIRCUITO VIRTUAL con garantías. Los recursos
los decide el router de INGRESO según la tabla de forwarding
(por eso MPLS se combina con DiffServ).
```

Los tres roles de LSR, y por qué el de tránsito es el que gana la velocidad:

```
┌──────────────┬──────────────────────┬────────────────────┬──────────────┐
│ INGRESS LSR  │ TRANSIT LSR          │ EGRESS LSR         │ ¿MIRA LA IP?│
├──────────────┼──────────────────────┼────────────────────┼──────────────┤
│ Examina los  │ Reenvía usando SÓLO  │ Remueve el         │ Ingress: SÍ  │
│ paquetes IP  │ la etiqueta, haciendo│ encabezado MPLS    │ Egress: SÍ   │
│ entrantes y  │ SWAPPING de etiqueta │ y pone el de capa  │ Transit: NO  │
│ los clasifica│ (label swap).        │ 3 de abajo.        │              │
│ asignándolos │ NO consulta la IP.   │                    │              │
│ a una FEC    │ NO consulta longest  │                    │              │
│ Genera el    │ prefix match.        │                    │              │
│ encabezado   │                      │                    │              │
│ MPLS y       │                      │                    │              │
│ asigna la    │                      │                    │              │
│ etiqueta     │                      │                    │              │
│ inicial      │                      │                    │              │
└──────────────┴──────────────────────┴────────────────────┴──────────────┘

⇦ LA VENTAJA DE MPLS, en una línea:
el transit LSR hace una BÚSQUEDA EXACTA de 20 bits (que el ASIC
puede hacer a 100s de Gbps) en vez de un LONGEST PREFIX MATCH de
32 bits en una TCAM (que es mucho más caro en silicio y).
```

Lo que hay que agregar y no está en la solución oficial de 2006 (y es el contenido de "Actualización a 2026"): MPLS **sigue desplegándose masivamente en los carriers**, pero su plano de control se está moviendo a **Segment Routing** (`RFC 8402` ; SR-MPLS `RFC 8660` ; SRv6 `RFC 8986` , que codifica los SIDs en la cabecera IPv6 y **elimina el MPLS por completo**). Y en datacenter, MPLS fue reemplazado por **EVPN-VXLAN** (`RFC 7432` + `RFC 7348`).

> Tope para 5 puntos: los **cuatro campos** de la etiqueta con su función (Label = índice en tabla, Experimental = QoS, BS = stacking, TTL = como en IP), la definición de **LSP** (túnel unidireccional de saltos controlados por etiquetas) y los **tres roles** con su verbo. Si además decís **por qué el de tránsito es el que da velocidad** (búsqueda exacta vs. longest prefix match), completaste.

PARTE 4 — CIERRE

📌 Resumen

1. **Internet es best effort**: reenvía lo que llega, descarta lo que no entra. Una videollamada y una descarga viajan igual. El problema real no es el ancho de banda sino el **buffer** de los routers (y el **bufferbloat**).
2. **Leaky bucket**: `b` = capacidad, `r` = tasa de salida. Si el paquete no entra, **se descarta**. Sale a ritmo **constante**: ráfagas imposibles.
3. **Token bucket**: los tokens se generan a `r` /s con tope `b`; un paquete consume `L` tokens. Si no hay, **espera**. Permite **ráfagas de hasta** `b`.
4. **La frase que hay que saber:** "*leaky = descarta, token = espera; leaky = constante, token = ráfaga*".
5. **Shaping** encola y retrasa (agrega latencia). **Policing** descarta o remarkea (no agrega latencia). La arquitectura estándar es **policing en la entrada, shaping en la salida**.
6. **IntServ** (`RFC 2211/2212/2213`): reserva por **flujo**, con **RSVP**, estado por flujo en cada router. Garantías reales, pero **no escala**. Hoy ≈ no se despliega (salvo RSVP-TE, `RFC 3209`, para traffic engineering de MPLS).
7. **DiffServ** (`RFC 2474` el campo DS, `RFC 2475` la arquitectura): clasifica en el **edge**, marca, y aplica el **PHB** en el **core**. Sin señalización, sin estado por flujo. **Es el que ganó.**
8. El campo de marcado es el **byte TOS / DS** de IPv4: **6 bits de DSCP + 2 bits de ECN**, en ese orden. PHB: **Default/BE**, **EF** (voz), **AF11..AF43**, **CS**.
9. **MPLS**: la **etiqueta** (20 bits de label, 3 de TC, 1 de S, 8 de TTL) es un **índice en la tabla del router**, no una dirección. El **LSP** es un camino predefinido **unidireccional** de saltos controlados por etiquetas.
10. Los **tres roles**: **ingress** (LER: mira la IP, deduce la FEC, empuja la etiqueta), **tránsito** (LSR: **sólo cambia etiquetas**, no mira la IP — ésa es la ganancia), **egreso** (LER: saca la etiqueta y pone la de capa 3).
11. **Fragmentación transparente**: cada router fragmenta, el destino reensambla, **cero estado**. **No transparente**: cada router reensambla antes de retransmitir, **hay que bufferizar**. MPLS es el sucesor eficiente de las dos.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. Explicame la diferencia entre leaky bucket y token bucket en una frase, y decime cuál de los dos **descarta**.

▶ Respuesta

2. Con un leaky bucket de `b = 100 bytes` y `r = 10 bytes/s`, ¿cuánto tarda la cubeta en vaciarse? ¿Y cada cuánto sale un paquete de 20 bytes?

▶ Respuesta

3. ¿Por qué MPLS es **más rápido** que el forwarding por IP?

▶ Respuesta

4. ¿Qué tiene que hacer un **edge router** de DiffServ y qué tiene que hacer un **core router**? ¿Y qué campo de IPv4 se usa para marcar?

▶ Respuesta

5. Escribí el layout del byte DS y decime qué PHB le pondrías a una descarga de FTP y a una llamada de voz.

▶ Respuesta

6. ¿Qué diferencia hay entre **IntServ** y **DiffServ**, y por qué uno ganó?

▶ Respuesta

7. ¿Por qué la arquitectura estándar es "policing en la entrada y shaping en la salida"?

▶ Respuesta

8. En el ejercicio del LSP, ¿qué etiqueta lleva el paquete en cada tramo, y qué miró cada router?

▶ Respuesta

🔗 Referencias

- **Examen `eirc0712` / `sirc0712`** — P1 (10 pts, leaky bucket vs. token bucket), P4 (10 pts, fragmentación transparente vs. no transparente), P5 (5 pts, router de ingreso y egreso en MPLS), P3 (10 pts, conmutación, ya resuelto en la Clase 4)
- **Examen `eirc0903` / `sirc0903`** — P4 (10 pts, DiffServ: edge, core y campo de marcado)
- **Examen `eirc1007` / `sirc1007`** — P4 (8 pts, paradigma de DiffServ)
- **Examen `eirc0608` / `sirc0608`** — P5 (5 pts, etiqueta, LSP y roles de MPLS)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 §5.6 (control de congestión y control de calidad de servicio) — *leaky bucket / token bucket*
- **RFC 2211 / 2212 / 2213** — IntServ (Integrated Services), 1997 · **RFC 3209** — RSVP-TE
- **RFC 2474** — el campo DS (Differentiated Services) · **RFC 2475** — arquitectura DiffServ
- **RFC 3246** — PHB EF (Expedited Forwarding) · **RFC 2597** — PHB AF (Assured Forwarding) · **RFC 4594** — guidelines de clases de servicio · **RFC 8622** — PHB LE (obsolesce RFC 3662)
- **RFC 3031** — MPLS architecture · **RFC 3032** — MPLS label stack · **RFC 4364** — MPLS VPN
- **RFC 8402** — Segment Routing (arquitectura) · **RFC 8660** — SR-MPLS · **RFC 8986** — SRv6 (elimina el MPLS)
- **RFC 7432** — EVPN · **RFC 7348** — VXLAN

- **Clase siguiente:** 46 — Gestión de redes: SNMP, NETCONF y YANG